

Analyzing True and False Positives Produced by JSpecify and NullAway

CS 6888: Software Analysis Project Presentation
Eric Weng

Java Null Safety

2025 CWE Top 25 Most Dangerous Software Weaknesses

13

NULL Pointer Dereference

[CWE-476](#) | CVEs in KEV: 0 | Rank Last Year: 21 (up 8) ▲

// Plain Java

```
Course myCourse = null;  
myCourse.getName();  
// NullPointerException
```

// Java Optionals

```
Optional<Course> myOpt = null;  
myOpt.orElse("none");  
// NullPointerException
```

JSpecify + NullAway

```
// 1. JSpecify annotations
```

```
@Nullable Course findCourse(@NonNull String courseCode) {  
    return /* Course object, or null*/;  
}
```

```
// 2. NullAway compilation
```

```
findCourse(null); // Passing @Nullable to @NonNull
```

```
findCourse("CS 101").getName(); // Possible @Nullable dereference
```

Worth the Opportunity Cost?

- Must migrate *existing* code
- Boy who cried “false positive”
- Why not just unit test?

akarnokd on Nov 22, 2018

Member

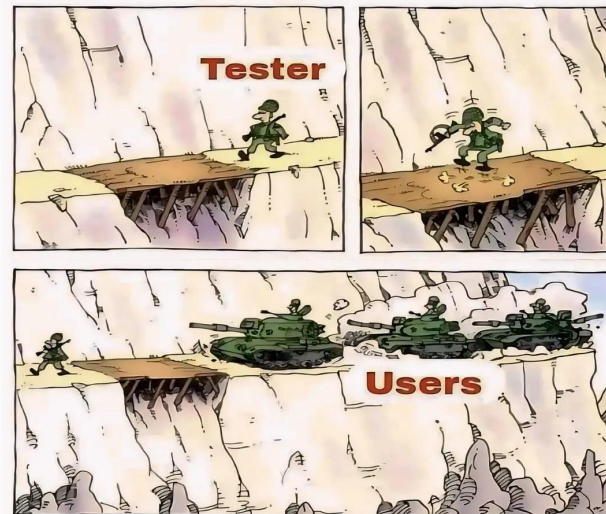
The key line here: `Task :compileJava FAILED`

The drawback of such tools I find is that they complain endlessly and you spend days and days to suppress and/or have clumsy workarounds to satisfy them.

RxJava works, compiles and passes its tests right now and there is no reason to worry about null dereferences or other presumed errors.

Previous Work

- No research papers on JSpecify
- 2019 NullAway paper
- Focus on performance/missed NPEs
- Low library domain diversity



NULLAWAY: Practical Type-Based Null Safety for Java

Subarno Banerjee
University of Michigan
Ann Arbor, MI, USA
subarno@umich.edu

Lazaro Clapp
Uber Technologies, Inc.
San Francisco, CA, USA
lazaro@uber.com

Manu Sridharan
University of California, Riverside
Riverside, CA, USA
manu@cs.ucr.edu

Approach



JSpecify Annotations

An artifact of well-named and well-specified annotations to power static analysis

Overview

Versions (7)

Used By (3.7k)

BOMs (177)

Badges

Artifacts using JSpecify Annotations (3,660)

- **Java group** 127 warnings
 - **Assignment issues group** 45 warnings
 - **Probable bugs group** 82 warnings
 - **Nullability problems group** 68 warnings
 - **@NotNull/@Nullable problems inspection** 7 warnings
 - **Return of 'null' inspection** 61 warnings
 - **Nullability and data flow problems inspection** 4 warnings
 - **Reference checked for 'null' is not used inside 'if' inspection** 10 warnings

```
[ERROR] COMPILATION ERROR :
```

```
[INFO] -----
```

```
[ERROR] /Users/ericweng/Documents/Schoolwork/SoftwareAnalysis/project/jsoup/src/main/java
/org/jsoup/nodes/Document.java:[190,9] [NullAway] returning @Nullable expression from
method with @NonNull return type (Add @SuppressWarnings)
```

Benchmark Libraries

Library	Category	Packages	Size (LoC)	Annotations
compile-testing	Compilation	(all)	~5,000	~90
guava	Utilities	base	~9,000	~50
jsoup	HTML Parser	parser	~14,000	~360
log4j-core	Logging	core, core.impl	~8,000	~50
Average	—	—	~9,000	~140

Performance of Analyzers

Warning	Unannotated	Annotated
False positive	114	120
True positive	17	23
Total	131	143
% False positive	87.0%	83.9%

- `@NonNull` by default
- IDE: Assignments and returns
- NullAway: Null dereferences
- Domain does not affect warnings

Top False Positives

1. Nullable variable not annotated

```
private /*@Nullable*/ instructor = null;
```

2. Unannotated method overrides

```
abstract void assign(@Nullable Instructor instructor);  
@Override void assign(Instructor instructor) {}
```

3. Ignored assertions

```
if (getCourse() != null) getCourse().assign(instructor);
```



Top True Positives

1. Null-marked method returns null

```
Instructor getInstructor(); // Interface
// Implementation class
@Override Instructor getInstructor() { return null; }
```

2. Dereference without null check

```
this.instructor = null; // Line 50
/* ... */
this.instructor.getName(); // Line 200
```

Conclusion

- Already safe code → trivial warnings
- Helps when no tests available
- Let code speak for itself
- Portability and standardization
- Wider adoption → safer together

End